



Linked Lists

Deep Dive + Real-World Scenarios + Complexity

Java 8+

THIRAVUGAL



Session Structure

1. Why Linked Lists?
2. Memory Model
3. Types of Linked Lists
4. Core Operations + Complexity
5. Fast & Slow Pointer Patterns
6. Real-world use cases
7. Interview-level problems



Why Linked Lists?

Arrays:

- Fixed size
- Contiguous memory
- $O(n)$ insertion in middle

Linked Lists:

- Dynamic size
- Non-contiguous memory
- $O(1)$ insertion at head



Linked List Memory Model

Each node contains:

- Data
- Pointer (reference) to next node

Memory layout:

Node → Node → Node → null

Not stored contiguously.

Node Structure (Java)

```
class Node {  
    int data;  
    Node next;  
  
    Node(int data) {  
        this.data = data;  
        this.next = null;  
    }  
}
```



Types of Linked Lists

1. Singly Linked List
2. Doubly Linked List
3. Circular Linked List



Singly Linked List

Structure:

Head $\rightarrow$ A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ null

Only forward traversal.

THIRAVUGAL



Doubly Linked List

Each node contains:

- prev pointer
- next pointer

Structure:

null ← A ↔ B ↔ C → null



Circular Linked List

Last node points back to head.

Useful in:

- Round-robin scheduling
- Multiplayer turn systems

Core Operations & Complexity

Operation	Time
Insert at head	$O(1)$
Insert at tail	$O(n)$
Delete at head	$O(1)$
Search	$O(n)$
Access by index	$O(n)$

Space Complexity: $O(n)$



Insert at Head

```
static Node insertAtHead(Node head, int data) {  
    Node newNode = new Node(data);  
    newNode.next = head;  
    return newNode;  
}
```

Time: $O(1)$



Insert at Tail

```
static Node insertAtTail(Node head, int data) {  
    Node newNode = new Node(data);  
    if (head == null) return newNode;  
  
    Node current = head;  
    while (current.next != null)  
        current = current.next;  
  
    current.next = newNode;  
    return head;  
}
```

Time: $O(n)$

Delete a Node

```
static Node delete(Node head, int key) {  
    if (head == null) return null;  
  
    if (head.data == key)  
        return head.next;  
  
    Node current = head;  
    while (current.next != null && current.next.data != key)  
        current = current.next;  
  
    if (current.next != null)  
        current.next = current.next.next;  
  
    return head;  
}
```

Time: $O(n)$



Traversal

```
static void printList(Node head) {  
    Node current = head;  
    while (current != null) {  
        System.out.print(current.data + " ");  
        current = current.next;  
    }  
}
```

Time: $O(n)$



Real World Use Case 1 — Music Playlist

Each song points to next.

Insert song quickly at head:

$O(1)$



Real World Use Case 2 — Undo/Redo Systems

Each action stored as node.

Move backward/forward using pointers.



Fast & Slow Pointer Pattern

Used for:

- Detect cycle
- Find middle
- Remove nth from end

Detect Cycle (Floyd's Algorithm)

Time: $O(n)$

Space: $O(1)$

```
static boolean hasCycle(Node head) {  
    Node slow = head;  
    Node fast = head;  
  
    while (fast != null && fast.next != null) {  
        slow = slow.next;  
        fast = fast.next.next;  
        if (slow == fast) return true;  
    }  
    return false;  
}
```

Find Middle of Linked List

Time: $O(n)$

Space: $O(1)$

```
static Node findMiddle(Node head) {  
    Node slow = head;  
    Node fast = head;  
  
    while (fast != null && fast.next != null) {  
        slow = slow.next;  
        fast = fast.next.next;  
    }  
    return slow;  
}
```



Reverse Linked List

Time: $O(n)$

Space: $O(1)$

```
static Node reverse(Node head) {  
    Node prev = null;  
    Node current = head;  
  
    while (current != null) {  
        Node next = current.next;  
        current.next = prev;  
        prev = current;  
        current = next;  
    }  
    return prev;  
}
```

Merge Two Sorted Lists

Time: $O(n + m)$

Space: $O(1)$

```
static Node merge(Node l1, Node l2) {  
    Node dummy = new Node(0);  
    Node current = dummy;  
  
    while (l1 != null && l2 != null) {  
        if (l1.data < l2.data) {  
            current.next = l1;  
            l1 = l1.next;  
        }  
    }
```



```
    else {  
        current.next = l2;  
        l2 = l2.next;  
    }  
    current = current.next;  
}  
  
current.next = (l1 != null) ? l1 : l2;  
return dummy.next;  
}
```

Space vs Array Comparison

Feature	Array	Linked List
Memory	Contiguous	Scattered
Insert Middle	$O(n)$	$O(1)^*$
Access by index	$O(1)$	$O(n)$
Cache Friendly	Yes	No

*If node reference is known.



Common Mistakes

- Forgetting null checks
- Losing head reference
- Infinite loops
- Not handling single-node lists
- Confusing reference updates

Interview Discussion Points

- When to choose Linked List?
- Why poor cache performance?
- Difference between singly and doubly?
- Why cycle detection works?



Practice Problems

Easy:

- Remove duplicates
- Delete nth node

Medium:

- Reverse in k-groups
- Detect cycle start

Advanced:

- Copy list with random pointer
- Flatten multilevel linked list



Summary

Linked Lists:

- Dynamic size
- Pointer-based structure
- $O(1)$ insertion at head
- Excellent for fast pointer patterns

Master:

- Reverse
- Cycle detection
- Middle finding
- Merging